

	Name (Field 1)	Age (Field 2)	Department (Field 3)
Record 1	Ann	21	CSE
Record 2	Manu	23	EEE
Record 3	Amy	24	CSE
Record 4	Jack	21	ECE

Figure 1: A file with four records, each with three fields

it is present on your system or not. In case you don't have AWK, then install it on Debian based GNU/Linux using the Advanced Package Tool (APT) package manager, as follows:

```
sudo apt-get install gawk
```

AWK is used for stream processing, where the basic unit is the string. It considers a text file as a collection of fields and records. Each row is a record, and a record is a collection of fields.

The syntax of AWK is as follows.
On the command line:

```
awk [options] 'pattern {action}' input file
```

As an AWK script:

```
awk [options] script_name input file
```

The most commonly used command-line options of `awk` are `-F` and `-f`:

```
-F : to change input field separator
-f : to name script file
```

A basic AWK program consists of patterns and actions — if the pattern is missing, the action is applied to all lines, or else, if the action is missing, the matched line is printed. There are two types of buffers used in AWK — the record buffer and field buffer. The latter is denoted as `$1`, `$2`... `$n`, where 'n' indicates the field number in the input file, i.e., `$` followed by the field number (so `$2` indicates the second field). The record buffer is denoted as `$0`, which indicates the whole record.

For example, to print the first field in a file, use the following command:

```
awk '{print $1}' filename
```

To print the third and first field in a file, use the command given below:

```
awk '{print $3, $1}' filename
```

AWK process flow

So how does one write an AWK script?

AWK scripts are divided into the following three parts — BEGIN (pre-processing), body (processing) and END (post-processing).

BEGIN (begins actions)
Patterns (Actions)
END (ends actions)

BEGIN is the part of the AWK script where variables can be initialised and report headings can be created. The processing body contains the data that needs to be processed, like a loop. END or the post-processing part analyses or prints the data that has been processed.

Let's look at an example for finding the total marks and averages of a set of students.

The AWK script is named as *awkscript*.

```
#Begin Processing
```

```
BEGIN {print "To find the total marks & average"}
{
```

```
#body processing
```

```
tot=$2+$3+$4
avg=tot/3
print "Total of " $1 ":" tot
print "Average of " $1 ":" avg
```

```
}
#End processing
```

```
END{print "...Script Finished..."}
```

Input file is named as *awkfile*

Input file (awkfile)

Aby 20 21 25

Amy 22 23 20

Running the awk script as :

```
awk -f awkscript awkfile
```

Output

To find the total marks & average

Total of Aby is : 66

Average of Aby is : 22

Total of Amy is : 65

Average of Amy is : 21.66

Classification of patterns

Expressions: AWK requires two operators while writing